

Resumo Extenso: Do Repositório Local ao GitHub

O fluxo de versionamento exige que você compreenda que seu trabalho acontece em dois espaços distintos, mas complementares: o **ambiente local** (sua máquina, onde você usa o Git) e o **ambiente remoto** (a plataforma colaborativa, o GitHub).

1. Criação e Inspeção (A Fundação Local) Antes de enviar qualquer coisa para a nuvem, o trabalho começa localmente. Você inicia um repositório do zero com `git init` ou copia um existente com `git clone`. A partir daí, a **inspeção** constante é sua melhor amiga: o comando `git status` deve ser usado o tempo todo para mapear o que foi alterado, enquanto o `git diff` mostra exatamente quais linhas sofreram modificações.

2. O Registro de Alterações e o "Commit Atômico" O Git separa as alterações em duas etapas: primeiro você seleciona o que vai ser salvo com o `git add` (colocando o arquivo na *staging area*), e depois consolida essa alteração no histórico com o `git commit`.

- **O que é um commit atômico?** É uma regra de ouro cobrada no seu quiz. Um commit atômico é um **commit pequeno e coerente sobre uma unidade lógica**. Ele não mistura uma correção de bug com a criação de um layout novo. Cada commit deve ter um propósito único.

3. Higiene e Segurança: O papel do .gitignore Você não deve enviar todos os arquivos da sua máquina para o GitHub. O arquivo `.gitignore` serve exatamente para **evitar o versionamento de arquivos indevidos**, como senhas (`.env`), arquivos temporários, pastas pesadas de dependências (`node_modules/`) ou logs. Configurar o `.gitignore` antes de fazer os commits garante um repositório limpo e seguro.

4. Sincronização Local-Remota (A Comunicação com o GitHub) É aqui que o repositório local conversa com o remoto. O termo **remote** (geralmente chamado de `origin`) é a ponte entre sua máquina e a URL do GitHub.

- **Comando de verificação:** Para saber exatamente para onde seu código está indo, você usa o `git remote -v`, que **mostra as URLs dos remotes configurados**.
- **O que o push realmente faz:** Quando você executa o `git push`, ele não envia arquivos soltos ou alterações não salvas; o `git push` envia estritamente os **commits locais** para o GitHub.
- **O uso estratégico do Fetch:** Se você quer ver se algum colega atualizou o código no GitHub, mas tem medo de bagunçar o seu trabalho atual, use o `git fetch`. Esse comando **baixa informações remotas sem alterar o diretório de trabalho**. Já o `git pull` é mais agressivo, pois ele baixa e tenta mesclar (*merge*) as alterações automaticamente.

5. Comunicação Profissional: Conventional Commits Um bom histórico de projeto conta uma história. O padrão *Conventional Commits* exige que a mensagem siga a estrutura `<tipo>(escopo opcional): <descrição>`. Uma mensagem perfeita, que inclusive é resposta do seu quiz, é: **docs: adiciona README inicial**. Mensagens genéricas como "alterei coisas" ou "update" dificultam a manutenção e não seguem o padrão.

Casos Comuns de Erros e Como Resolvê-los

O Git possui ferramentas de recuperação para diferentes estágios do seu erro.

1. O Push foi rejeitado pelo GitHub

- **Por que acontece?** Isso ocorre porque o repositório remoto tem commits que o seu repositório local não possui. O Git bloqueia o envio para evitar que você apague o trabalho de outra pessoa.
- **Como resolver:** Faça um diagnóstico seguro. Use `git fetch origin` para ver o que há de novo, depois `git pull origin main` para integrar as alterações no seu código e, somente após resolver possíveis conflitos, use `git push`. Nunca use `git push --force` neste estágio.

2. Errei localmente, mas AINDA NÃO fiz o commit

- **Alteração indesejada:** Editou um arquivo e estragou o código? Use `git restore <arquivo>` para voltar ao estado do último commit.
- **Adicionou ao staging por engano:** Deu um `git add` no arquivo errado? Use `git restore --staged <arquivo>` para tirá-lo da fila do commit, sem perder o que você digitou.

3. Fiz o commit, mas ainda está local (não dei push)

- Use `git reset`. O `git reset --soft HEAD~1` desfaz o último commit, mas mantém seus arquivos intactos e no esquema de *staging*, prontos para você consertar a mensagem e commitar de novo.

4. Publiquei um commit com erro no GitHub (A Regra de Ouro)

- Se o erro já foi para o GitHub, outras pessoas podem ter baixado. Para desfazer um commit já publicado, a ferramenta preferencial e correta é o `git revert <hash>`. Ele cria um novo commit que desfaz a alteração anterior, preservando todo o histórico compartilhado de forma segura.

Tabela Prática de Comandos e Casos Ideais de Uso

Comando Git	O que faz	Caso Ideal de Uso
<code>git init</code>	Prepara uma pasta para controle de versão.	Quando você está começando um projeto totalmente do zero na sua máquina.

git clone	Copia repositório, histórico e configurações remotas.	Quando você precisa baixar um projeto que já existe no GitHub para trabalhar nele.
git status	Informa a branch atual e arquivos modificados/preparados.	Uso contínuo: execute sempre antes de adicionar, commitar ou recuperar erros para saber onde você está.
git add	Prepara arquivos para o próximo snapshot (staging).	Quando você terminou uma unidade lógica de código e quer selecioná-la para o commit.
git commit -m	Registra as alterações preparadas no histórico.	Para criar um "ponto de salvamento" atômico com uma mensagem clara sobre o que mudou.
git remote -v	Lista URLs de remotes configurados.	Quando você precisa conferir se o seu projeto local está apontando para o repositório correto no GitHub.
git push	Envia commits locais para o repositório remoto.	Ao finalizar uma parte do trabalho local que está pronta para ser publicada e compartilhada com a equipe.
git fetch	Baixa referências e objetos remotos sem alterar arquivos locais.	Para inspecionar mudanças feitas por colegas de forma 100% segura, sem mexer no seu código atual.

git pull	Baixa e integra alterações remotas (gerando merge se necessário).	Quando você conferiu o que há de novo no remoto e está pronto para atualizar seu trabalho local.
git restore	Restaura o arquivo ao estado do último commit.	Quando você bagunçou um arquivo não salvo e quer simplesmente recomeçar do zero a partir do último commit.
git reset	Move referências locais, desfazendo commits não publicados.	Quando você fez um commit ruim, mas ainda não mandou para o GitHub, e precisa reorganizar o histórico local.
git revert	Cria um commit inverso preservando histórico.	Quando um código quebrado já foi enviado (push) ao GitHub e você precisa anulá-lo sem apagar o histórico.

Glossário Essencial

- **Git:** Sistema distribuído de controle de versão que registra snapshots (fotografias) do estado do seu projeto.
- **GitHub:** Plataforma online utilizada para hospedar repositórios Git e apoiar a colaboração entre desenvolvedores.
- **Remote:** Nome (geralmente **origin**) associado à URL de um repositório remoto no GitHub.
- **Push:** Ação de enviar os seus commits locais para o repositório remoto.
- **Fetch:** Ação de baixar informações do repositório remoto de forma segura, sem alterar seu diretório de trabalho local.
- **Pull:** Operação composta que executa um **fetch** e imediatamente integra as alterações remotas na sua branch local.
- **Commit:** Registro histórico de uma unidade lógica de mudança no código.
- **.gitignore:** Arquivo de texto que define padrões de arquivos que o Git deve ignorar e nunca rastrear.
- **Reset:** Comando de recuperação que reorganiza ou desfaz commits locais que ainda não foram publicados.
- **Revert:** Comando que cria um novo commit com o propósito exclusivo de desfazer os efeitos de um commit anterior (ideal para histórico já compartilhado).
- **Conventional Commits:** Padrão profissional de escrita de mensagens de commit (ex: **tipo(escopo): descrição**).

Quiz de Fixação: Git e GitHub (Módulo 2)

Questões de Múltipla Escolha

1. Qual comando é utilizado para iniciar um novo repositório Git em uma pasta local a partir do zero?

- A) `git clone`
- B) `git init`
- C) `git remote add`
- D) `git status`

2. Qual é a diferença principal entre os comandos `git init` e `git clone`?

- A) `git init` baixa um projeto do GitHub; `git clone` cria uma pasta em branco.
- B) `git init` prepara uma pasta local do zero; `git clone` copia um repositório existente, seu histórico e configurações remotas.
- C) `git init` serve apenas para salvar arquivos; `git clone` envia alterações.
- D) Ambos realizam exatamente a mesma função sem diferenças técnicas.

3. Para verificar o estado atual da branch, identificar arquivos modificados ou saber o que está na *staging area*, o comando indicado é:

- A) `git diff`
- B) `git log`
- C) `git status`
- D) `git remote -v`

4. Qual é a função do comando `git diff` no fluxo de inspeção?

- A) Exibir a lista de commits já realizados no projeto.
- B) Mostrar exatamente quais linhas de código sofreram modificações nos arquivos.
- C) Listar todos os arquivos ignorados pelo `.gitignore`.
- D) Sincronizar o repositório local diretamente com o GitHub.

5. O que é um "commit atômico"?

- A) Um commit extenso contendo todo o trabalho de uma semana de projeto.
- B) Um commit feito diretamente na interface do GitHub sem utilizar o terminal local.
- C) Um commit pequeno e coerente focado em uma única unidade lógica de mudança.
- D) Um commit que apaga o histórico do projeto para economizar memória.

6. Ao executar o comando `git add <arquivo>`, qual etapa do fluxo de trabalho é executada?

- A) O arquivo é enviado diretamente para a nuvem no GitHub.
- B) O arquivo é incluído na *staging area* (fila de preparação para o próximo commit).
- C) O arquivo é removido permanentemente do rastreamento de versão.
- D) O snapshot é gravado de forma definitiva no histórico local.

7. O que acontece quando o comando **git commit** é executado com sucesso?

- A) As alterações preparadas na *staging area* são consolidadas em um snapshot no histórico local.
- B) O código é baixado da nuvem e integrado à máquina local.
- C) O repositório local é conectado a uma nova URL remota.
- D) Os arquivos modificados voltam ao estado original do último salvamento.

8. Qual é a finalidade do arquivo **.gitignore** em um repositório?

- A) Armazenar senhas em texto puro para sincronização automática no GitHub.
- B) Evitar o versionamento de arquivos indevidos, como credenciais, logs e dependências pesadas.
- C) Aumentar a velocidade do comando **git push**.
- D) Definir os nomes das branches padrão do projeto.

9. Qual das opções abaixo indica arquivos ou pastas que **NUNCA** devem ser enviados ao GitHub e devem constar no **.gitignore**?

- A) O arquivo **README.md** do projeto.
- B) Arquivos de código-fonte principais (como **.c** ou **.java**).
- C) Arquivo de variáveis de ambiente com senhas (**.env**) e diretórios de dependências (**node_modules/**).
- D) Arquivos de documentação técnica e requisitos.

10. O termo **remote** (frequentemente nomeado como **origin**) representa:

- A) O diretório oculto **.git** no computador local.
- B) A ponte ou endereço de conexão entre a máquina local e a URL hospedada no GitHub.
- C) Uma ramificação temporária criada apenas para ajustar erros de sintaxe.
- D) O histórico de mensagens salvas localmente.

11. Qual comando permite visualizar as URLs dos repositórios remotos configurados na sua máquina?

- A) **git status**
- B) **git remote -v**
- C) **git fetch --list**
- D) **git push --show**

12. O que o comando **git push** envia efetivamente para o repositório no GitHub?

- A) Todos os arquivos editados no computador, mesmo os que não foram salvos no commit.
- B) Apenas as configurações salvas no **.gitignore**.
- C) Estritamente os commits já consolidados no histórico local.
- D) Somente os arquivos que se encontram na *staging area*.

13. Qual é a diferença fundamental entre os comandos **git fetch** e **git pull**?

- A) O `git fetch` envia alterações locais; o `git pull` apaga ramificações remotas.
- B) O `git fetch` baixa dados remotos sem alterar o diretório de trabalho; o `git pull` baixa e tenta mesclar as alterações no código local.
- C) O `git fetch` exclui o histórico local; o `git pull` cria novas branches.
- D) Ambos os comandos possuem o mesmo comportamento no Git.

14. Por que o `git fetch` é considerado uma operação 100% segura para inspecionar o repositório?

- A) Porque ele apaga arquivos temporários da máquina.
- B) Porque ele força a substituição de todo o código local pelo código do servidor.
- C) Porque ele busca as atualizações do servidor sem alterar os seus arquivos de trabalho locais.
- D) Porque ele impede que novos commits sejam feitos na branch.

15. O comando `git pull` realiza internamente a junção de quais duas operações?

- A) `git add` e `git commit`
- B) `git fetch` e a integração/mesclagem (`merge`)
- C) `git init` e `git clone`
- D) `git status` e `git push`

16. Assinale a alternativa que segue corretamente a especificação do *Conventional Commits*:

- A) "arrumei alguns arquivos e criei a documentação"
- B) `docs: adiciona README inicial`
- C) "UPDATE DEFINITIVO DO PROJETO - V2"
- D) `commit #102 - atualizacoes`

17. Qual é a vantagem de adotar o padrão *Conventional Commits* nos projetos de software?

- A) Aumentar a velocidade do terminal de comandos.
- B) Manter o histórico de alterações limpo, organizado e compreensível para toda a equipe.
- C) Eliminar a necessidade de usar o comando `git add`.
- D) Impedir qualquer tipo de conflito entre branches.

18. Por qual motivo o GitHub pode rejeitar a execução de um `git push` feito pela máquina local?

- A) Porque o arquivo `.gitignore` foi excluído da pasta local.
- B) Porque o repositório remoto possui commits novos que a sua máquina local ainda não baixou.
- C) Porque foram realizados mais de três commits no mesmo dia.
- D) Porque o terminal não executou o `git status` antes.

19. Qual é o fluxo seguro para resolver um "Push Rejeitado" decorrente de atualizações no repositório remoto?

- A) Usar `git push --force` para sobrescrever o código do GitHub.
- B) Apagar a pasta do projeto local e fazer um novo clone.
- C) Usar `git fetch` para verificar as novidades, integrar com `git pull` (resolvendo eventuais conflitos) e realizar o `git push`.
- D) Reiniciar o serviço do Git com `git remote reset`.

20. Caso você edite um arquivo localmente e queira descartar as alterações não commitadas (voltando ao estado do último commit), o comando a ser usado é:

- A) `git restore <arquivo>`
- B) `git revert <arquivo>`
- C) `git reset <arquivo>`
- D) `git remove <arquivo>`

21. Se você adicionou por engano um arquivo na *staging area* (`git add`), como pode removê-lo do staging sem perder as linhas de código digitadas?

- A) `git restore <arquivo>`
- B) `git restore --staged <arquivo>`
- C) `git revert --staged <arquivo>`
- D) `git rm -f <arquivo>`

22. Para desfazer o último commit realizado apenas na sua máquina LOCAL (sem apagar as alterações do código e mantendo-as no staging), utiliza-se:

- A) `git revert HEAD`
- B) `git reset --soft HEAD~1`
- C) `git push --delete`
- D) `git checkout --clean`

23. Qual é a ferramenta recomendada para desfazer as alterações de um commit que JÁ FOI PUBLICADO no GitHub e compartilhado com outros desenvolvedores?

- A) `git reset --hard`
- B) `git revert <hash>`
- C) `git clear remote`
- D) `git delete commit`

24. Por que o `git revert` é o comando correto para tratar commits que já foram enviados ao repositório público no GitHub?

- A) Porque ele apaga o histórico do servidor sem deixar vestígios.
- B) Porque ele cria um novo commit que desfaz o efeito do anterior, mantendo a integridade e a história pública intactas.
- C) Porque ele ignora os arquivos listados no `.gitignore`.
- D) Porque ele bloqueia o acesso de outros desenvolvedores à branch principal.

25. O que ocorre se você tentar executar `git commit -m "mensagem"` sem ter adicionado nada à *staging area* via `git add`?

- A) O Git cria um commit vazio no histórico.
- B) O Git exibe uma mensagem informando que não há alterações preparadas para o commit (*nothing added to commit*).
- C) O Git exclui todas as modificações não salvas do disco.
- D) O Git envia os arquivos direto para o servidor remoto.

26. Na mensagem de commit `feat(authenticacao): adiciona login com Google`, qual elemento representa o *tipo* da alteração segundo os Conventional Commits?

- A) `autenticacao`
- B) `feat`
- C) `adiciona login com Google`
- D) O caractere de dois-pontos :

27. Qual é a função da opção `--soft` junto ao comando `git reset`?

- A) Deletar permanentemente as alterações do disco rígido.
- B) Desfazer o commit mantendo todas as modificações de arquivos salvas na *staging area*.
- C) Alterar a mensagem do commit apenas no GitHub.
- D) Bloquear temporariamente a edição dos arquivos.

28. A sequência correta e lógica de comandos para preparar, gravar e publicar o seu código no GitHub é:

- A) `git push` -> `git commit` -> `git add`
- B) `git add` -> `git commit` -> `git push`
- C) `git init` -> `git pull` -> `git add`
- D) `git status` -> `git revert` -> `git push`

29. Inspecionar o repositório com `git status` antes de executar ações importantes ajuda a:

- A) Entender exatamente em qual branch você está e qual o estado atual dos arquivos, prevenindo erros no commit ou restauração.
- B) Apagar automaticamente os arquivos temporários criados pelo sistema operacional.
- C) Testar a velocidade da sua conexão com os servidores do GitHub.
- D) Atualizar a versão do software Git instalado no computador.

30. Do ponto de vista conceitual, o Git e o GitHub definem-se respectivamente como:

- A) Uma linguagem de programação e um banco de dados relacional.
- B) Um sistema distribuído de controle de versão local e uma plataforma online para hospedagem e colaboração de código.
- C) Um editor de texto para código e um servidor de hospedagem web.
- D) Um sistema operacional e um serviço genérico de armazenamento em nuvem.

Gabarito Oficial

Questão	Resposta Correta	Tópico do Módulo 2
1	B	Inicialização de repositório local (<code>git init</code>)
2	B	Diferença entre <code>git init</code> e <code>git clone</code>
3	C	Inspeção com <code>git status</code>
4	B	Visualização de alterações com <code>git diff</code>
5	C	Conceito de Commit Atômico
6	B	Uso do <code>git add</code> e a Staging Area
7	A	Registro do snapshot local com <code>git commit</code>
8	B	Finalidade do <code>.gitignore</code>
9	C	Segurança e arquivos indevidos no versionamento
10	B	Conceito de Remote / <code>origin</code>
11	B	Listagem de remotes (<code>git remote -v</code>)
12	C	Envio de commits locais via <code>git push</code>

13	B	Diferença entre <code>git fetch</code> e <code>git pull</code>
14	C	Segurança na utilização do <code>git fetch</code>
15	B	Operações internas do <code>git pull</code>
16	B	Aplicação prática de Conventional Commits
17	B	Benefícios do Conventional Commits
18	B	Causa do erro de Push Rejeitado
19	C	Solução recomendada para Push Rejeitado
20	A	Descarte de alterações não salvos com <code>git restore</code>
21	B	Remoção da Staging Area (<code>git restore --staged</code>)
22	B	Desfazer commit local mantendo arquivos (<code>git reset --soft</code>)
23	B	Reversão de commits públicos (<code>git revert</code>)
24	B	Preservação do histórico público com <code>git revert</code>
25	B	Comportamento do commit sem staged files

26	B	Estrutura das mensagens de commit (Tipo)
27	B	Funcionamento da flag <code>--soft</code> do <code>reset</code>
28	B	Sequência lógica do fluxo de versionamento (<code>add</code> -> <code>commit</code> -> <code>push</code>)
29	A	Boas práticas de inspeção contínua
30	B	Conceitos fundamentais de Git vs GitHub